

OE-EC704B

Optimization Technique

Combined Module Notes

Modules 1 – 6

Semester VII • 3L:0T:0P • 3 Credits

CGEC | B.Tech ECE (2023–27)

Biraj Sarkar

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Contents

Module 1: Introduction to Optimization	4
1 Introduction to Optimization	4
1.1 Standard Mathematical Formulation	4
1.2 Key Design Elements	4
2 Engineering Optimization Problem Formulation	5
2.1 Classic Example: Design of a Cylindrical Water Tank	5
3 Classification of Optimization Problems	5
4 Classification of Optimization Algorithms	6
4.1 The Optimization Design Loop	6
Module 2: Optimality Criteria for Single-Variable Functions	10
1 Optimality Criteria for Single-Variable Functions	10
1.1 Necessary Condition (First-Order)	10
1.2 Sufficient Condition (Second-Order)	10
2 Bracketing Methods	11
2.1 Exhaustive Search Method	11
3 Region-Elimination Methods	11
3.1 Working Principle of Region Elimination	11
3.2 Interval Halving Method	12
3.3 Fibonacci Search Method	12
4 Point Estimation Methods	13
4.1 Successive Quadratic Estimation (SQE)	13
Module 3: Fundamentals of Gradient-Based Search	16
1 Fundamentals of Gradient-Based Search	16
2 Newton-Raphson Method	16
2.1 Derivation of the Iterative Formula	16
2.2 Convergence and Characteristics	17
3 Bisection Method	17
3.0.1 Algorithm Steps	17
4 Secant Method	18
4.1 Iterative Formula	18
5 Computer Programs / Pseudo-Codes	18
Module 4: Optimality Criteria for Multivariable Functions	22
1 Optimality Criteria for Multivariable Functions	22
1.1 Necessary Condition (First-Order)	22

1.2	Sufficient Condition (Second-Order)	22
2	Direct Search Methods (No Gradients)	23
2.1	Simplex Search Method (Nelder-Mead)	23
2.2	Hooke-Jeeves Pattern Search Method	23
3	Gradient-Based Descent Methods	23
3.1	Cauchy's (Steepest Descent) Method	23
3.2	Newton's Multivariable Method	24
4	Multi-Objective and Pareto Optimization	24
4.1	Pareto Dominance & Optimality	24
Module 5: Characteristics of Constrained Problems		27
1	Characteristics of Constrained Problems	27
2	Direct Constrained Optimization Methods	27
2.1	Box's Complex Method	28
2.2	Kelly's Cutting Plane Method	28
3	Indirect Methods (Transformation Techniques)	28
3.1	Exterior Penalty Function Method	28
3.2	Interior Penalty (Barrier) Function Method	29
4	Convex Optimization Methods	29
Module 6: Genetic Algorithm (GA)		32
1	Genetic Algorithm (GA)	32
1.1	Basic Genetic Operators	32
1.2	The Genetic Algorithm Flowchart	32
2	Simulated Annealing (SA)	33
2.1	The Metropolis Acceptance Criterion	33
3	Swarm Intelligence Algorithms	34
3.1	Particle Swarm Optimization (PSO)	34
3.2	Swarm Variants: DE, BFA, and ACO	34

Module 1

Introduction to Optimization

1. Introduction to Optimization

Optimization is the act of obtaining the best result under given circumstances. In engineering design, it refers to making decisions that maximize desired factors (such as efficiency, reliability, or strength) or minimize undesired factors (such as cost, weight, or energy loss).

1.1 Standard Mathematical Formulation

Every optimization problem can be formulated in a canonical mathematical representation:

Canonical Optimization Problem

Minimize the objective function:

$$f(\mathbf{x}) = f(x_1, x_2, \dots, x_n) \quad (1)$$

subject to:

$$g_j(\mathbf{x}) \leq 0, \quad j = 1, 2, \dots, m \quad (\text{Inequality Constraints}) \quad (2)$$

$$h_k(\mathbf{x}) = 0, \quad k = 1, 2, \dots, p \quad (\text{Equality Constraints}) \quad (3)$$

and the variable bounds (side constraints):

$$x_i^{(L)} \leq x_i \leq x_i^{(U)}, \quad i = 1, 2, \dots, n \quad (4)$$

Where $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ is the design vector.

1.2 Key Design Elements

- Design Vector ($\mathbf{x}$):** A set of n independent variables that uniquely define the design. These parameters are varied by the optimizer (e.g., length, thickness, resistance, voltage).
- Objective Function ($f(\mathbf{x})$):** A mathematical expression that quantifies the criterion to be optimized. If the objective is maximization, it is equivalent to minimizing $-f(\mathbf{x})$.
- Design Constraints:** Conditions that must be satisfied for a design to be acceptable.
 - Behavior Constraints:* Restraints on the functional behavior of the system (e.g., max deflection, voltage limit).

- *Geometric / Side Constraints*: Direct limits on physical dimensions (e.g., thickness $t \geq 2$ mm).

4. **Feasible Region**: The set of all design points in the design space that satisfy all equality and inequality constraints.

2. Engineering Optimization Problem Formulation

To optimize an engineering system, the physical problem must be translated into standard mathematical form.

2.1 Classic Example: Design of a Cylindrical Water Tank

Let us formulate the problem of designing a cylindrical water tank of a specified storage capacity (V_0) that minimizes the total manufacturing cost (material cost).

Cylindrical Tank Optimization Formulation

1. **Design Variables**: Let the radius of the tank be r and the height be h .

$$\mathbf{x} = [r, h]^T$$

2. **Objective Function**: To minimize the manufacturing cost, we minimize the total surface area (base area + cylindrical wall area):

$$\text{Minimize } f(r, h) = \pi r^2 + 2\pi r h \quad (5)$$

3. **Constraints**: The tank must hold a minimum volume V_0 :

$$\pi r^2 h \geq V_0 \implies V_0 - \pi r^2 h \leq 0$$

Hence, the inequality constraint is:

$$g_1(r, h) = V_0 - \pi r^2 h \leq 0 \quad (6)$$

Additionally, we have non-negativity side constraints:

$$r \geq 0, \quad h \geq 0 \quad (7)$$

3. Classification of Optimization Problems

Optimization problems are classified based on structure, constraints, and objective behaviors:

Parameter	Class A	Class B
Constraints	Unconstrained: No limits on design space; $\min f(\mathbf{x})$.	Constrained: Limits exist ($g_j(\mathbf{x}) \leq 0$, $h_k(\mathbf{x}) = 0$).
Linearity	Linear Programming (LP): f , g_j , and h_k are all linear functions of $\mathbf{x}$.	Non-linear Programming (NLP): At least one function is non-linear.
Variables	Single-variable: $f(x)$ depends on one parameter only.	Multivariable: $f(\mathbf{x})$ depends on two or more parameters.
Nature	Static: Design variables represent constant parameters.	Dynamic: Design variables are functions of time.

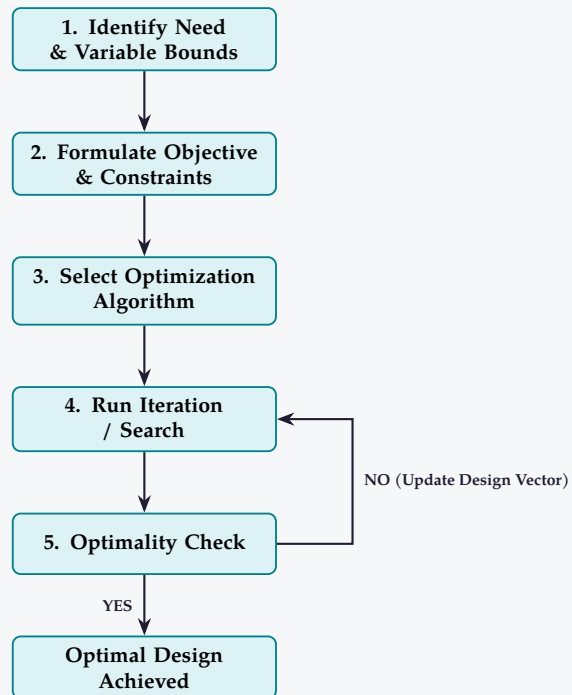
4. Classification of Optimization Algorithms

Optimization algorithms are structured into three primary groups based on search logic:

- I. Analytical Methods (Calculus-Based):** Uses derivative information ($f'(x)$, ∇f) to establish optimality conditions. Extremely fast but requires the objective function to be continuous and differentiable.
- II. Numerical Methods (Search-Based):** Iterative procedures that evaluate the objective function at sequence points to narrow down the optimum.
 - *Direct Search Methods:* Require objective function values only (e.g., Simplex, Hooke-Jeeves).
 - *Gradient-Based Methods:* Require function value and derivative values (e.g., Steepest Descent, Newton).
- III. Advanced / Meta-heuristic Algorithms:** Nature-inspired algorithms that explore global design spaces to bypass local minima traps (e.g., Genetic Algorithms, Particle Swarm Optimization, Simulated Annealing).

4.1 The Optimization Design Loop

The sequence of engineering design optimization is mapped below:

Optimization Design Process Flowchart

Quick Revision — Module 1

Mathematical Formulation & Fundamentals | OE-EC704B

Key Concepts & Mathematical Rules

Objective Function	$f(\mathbf{x}) \rightarrow$ function to be minimized or maximized.
Inequality Constraints	$g_j(\mathbf{x}) \leq 0, \quad j = 1, \dots, m.$
Equality Constraints	$h_k(\mathbf{x}) = 0, \quad k = 1, \dots, p.$
Side Constraints	$x_i^{(L)} \leq x_i \leq x_i^{(U)}, \quad i = 1, \dots, n.$
Design Vector	$\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ containing n design variables.

One-Line Definitions

- **Design Variable:** An independent parameter that can be varied during the design optimization process.
- **Behavior Constraint:** A constraint bounding the system behavior under operational loads (e.g., maximum stress).
- **Feasible Space:** The region of design space in which all inequality and equality constraints are fully satisfied.
- **Non-linear Programming (NLP):** An optimization problem containing non-linearities in objective or constraint functions.
- **Global Minimum:** The absolute lowest value of the objective function in the entire feasible design space.
- **Local Minimum:** The lowest value of the objective function within a local neighborhood of the design space.

Frequently Asked / Viva Questions

Q1. State the standard mathematical formulation of a constrained optimization problem.

Minimize $f(\mathbf{x})$ subject to $g_j(\mathbf{x}) \leq 0$ for $j = 1, \dots, m$, $h_k(\mathbf{x}) = 0$ for $k = 1, \dots, p$, and $x_i^{(L)} \leq x_i \leq x_i^{(U)}$ for $i = 1, \dots, n$.

Q2. Differentiate between design variables and design parameters.

Design variables are changed by the optimization algorithm to find the optimum design. Design parameters are constant coefficients fixed before the optimization starts (e.g., material density, young's modulus).

Q3. Differentiate between behavior constraints and side constraints.

Behavior constraints depend on the state variables of the system and require

Frequently Asked / Viva Questions (contd.)

system analysis to evaluate (e.g., $\text{stress} \leq \sigma_{allow}$). Side constraints directly bound the design variables (e.g., diameter $d \geq 10$ mm).

Q4. What is a feasible design space?

A feasible design space is the collection of all design vectors $\mathbf{x}$ that satisfy all constraints simultaneously.

Q5. What is the difference between single-objective and multi-objective optimization?

Single-objective optimization seeks to optimize a single criterion. Multi-objective optimization optimizes multiple conflicting objectives simultaneously, yielding a set of trade-off designs (Pareto front).

Q6. Why do we reformulate a maximization problem as a minimization problem?

To maintain algorithmic consistency. Any maximization problem can be solved using standard minimization algorithms by minimizing $-f(\mathbf{x})$.

Q7. What makes a problem a Linear Programming Problem (LPP)?

An optimization problem is an LPP if the objective function and all equality/inequality constraints are linear functions of the design variables.

Q8. Classify optimization methods based on their search logic.

Methods are classified into Analytical Methods (calculus-based), Numerical Methods (Search-based: Direct and Gradient-based), and Advanced Heuristic Methods (nature-inspired: GA, PSO, SA).

Q9. Explain the role of the objective function.

The objective function acts as a mathematical evaluator that measures how good a particular candidate design vector is relative to design goals.

Q10. What are the side constraints in cylinder volume maximization under surface area limits?

The side constraints are the physical non-negativity limits of the geometric parameters: Radius $r \geq 0$ and Height $h \geq 0$.

Module 2

Optimality Criteria for Single-Variable Functions

1. Optimality Criteria for Single-Variable Functions

Optimality criteria determine whether a candidate point x^* is a local minimum, local maximum, or an inflection point.

1.1 Necessary Condition (First-Order)

Theorem: Necessary Condition

If a function $f(x)$ is continuous and differentiable at x^* , and x^* is a local extremum (minimum or maximum), then the first derivative at x^* must vanish:

$$f'(x^*) = \left. \frac{df}{dx} \right|_{x=x^*} = 0 \quad (8)$$

Such a point x^* is called a **Stationary Point**.

1.2 Sufficient Condition (Second-Order)

Theorem: Sufficient Condition

Let $f(x)$ be twice differentiable at the stationary point x^* .

- If $f''(x^*) > 0$, then x^* is a **local minimum**.
- If $f''(x^*) < 0$, then x^* is a **local maximum**.
- If $f''(x^*) = 0$, the test is inconclusive. We must evaluate higher-order derivatives:

$$f'(x^*) = f''(x^*) = \dots = f^{(n-1)}(x^*) = 0, \quad f^{(n)}(x^*) \neq 0 \quad (9)$$

- If n is **odd**, x^* is an **inflection point** (neither minimum nor maximum).
- If n is **even** and $f^{(n)}(x^*) > 0$, x^* is a **local minimum**.
- If n is **even** and $f^{(n)}(x^*) < 0$, x^* is a **local maximum**.

2. Bracketing Methods

Bracketing methods find an initial interval of uncertainty containing the optimum before applying fast search routines.

2.1 Exhaustive Search Method

Exhaustive search evaluates a function at equally spaced points in an interval $[a, b]$ to find the sub-interval containing the minimum.

Exhaustive Search Algorithm

Let the initial interval be $[a, b]$, and let the number of evaluations be n .

1. Calculate the step size:

$$\Delta x = \frac{b - a}{n} \quad (10)$$

2. Define evaluation coordinates: $x_i = a + i \cdot \Delta x$, for $i = 0, 1, \dots, n$.

3. Evaluate $f(x_i)$ sequentially for $i = 1, 2, \dots, n - 1$.

4. Identify three consecutive points x_{k-1} , x_k , and x_{k+1} such that:

$$f(x_{k-1}) > f(x_k) \quad \text{and} \quad f(x_k) < f(x_{k+1}) \quad (11)$$

5. The minimum lies bracketed in the interval $[x_{k-1}, x_{k+1}]$. The new interval of uncertainty is:

$$L_1 = 2 \cdot \Delta x = \frac{2(b - a)}{n} \quad (12)$$

3. Region-Elimination Methods

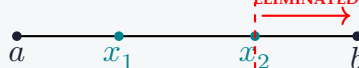
Region-elimination methods systematically reduce the interval of uncertainty by comparing objective values at test points.

3.1 Working Principle of Region Elimination

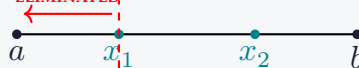
If we assume $f(x)$ is **unimodal** (has only one minimum in the interval $[a, b]$), we can eliminate a portion of the interval by evaluating two test points x_1 and x_2 (where $x_1 < x_2$):

Unimodal Region Elimination Logic

If $f(x_1) < f(x_2)$, minimum cannot lie in $(x_2, b]$



If $f(x_1) > f(x_2)$, minimum cannot lie in $[a, x_1)$



3.2 Interval Halving Method

This method eliminates exactly half of the interval at each stage using three evaluations (x_0 , x_1 , and x_2).

Interval Halving Algorithm

Let the current interval be $[a, b]$.

1. Calculate the midpoint: $x_0 = \frac{a+b}{2}$.

2. Calculate quarter points:

$$x_1 = a + \frac{b-a}{4}, \quad x_2 = b - \frac{b-a}{4} \quad (13)$$

3. Evaluate $f(x_1)$, $f(x_0)$, and $f(x_2)$.

4. Compare function values to eliminate region:

- If $f(x_1) < f(x_0)$, the new interval is $[a, x_0]$.
- If $f(x_2) < f(x_0)$, the new interval is $[x_0, b]$.
- If $f(x_1) \geq f(x_0)$ and $f(x_2) \geq f(x_0)$, the new interval is $[x_1, x_2]$.

5. In all cases, the new interval width is exactly:

$$L_{new} = \frac{b-a}{2} \quad (14)$$

3.3 Fibonacci Search Method

Fibonacci search is the mathematically optimal region-elimination method. It uses the Fibonacci sequence $F_0 = 1, F_1 = 1, F_n = F_{n-1} + F_{n-2}$ to calculate placement ratios.

Fibonacci Search Equations

For a pre-selected total number of function evaluations n :

1. The initial two test points x_1 and x_2 in $[a, b]$ are placed at:

$$x_1 = a + \frac{F_{n-2}}{F_n}(b-a) \quad (15)$$

$$x_2 = b - \frac{F_{n-2}}{F_n}(b-a) = a + \frac{F_{n-1}}{F_n}(b-a) \quad (16)$$

2. After region elimination, the remaining sub-interval is evaluated. At the k -th step, the new test point is placed at:

$$x_k = \text{Remaining Limit} \pm \frac{F_{n-k-1}}{F_{n-k+1}}(\text{Current Interval Width}) \quad (17)$$

3. The final reduction ratio of the interval of uncertainty after n iterations is:

$$\frac{L_n}{L_1} = \frac{1}{F_n} \quad (18)$$

4. Point Estimation Methods

Unlike region elimination, point estimation methods fit a smooth polynomial through known points to estimate the local minimum directly.

4.1 Successive Quadratic Estimation (SQE)

SQE fits a quadratic function $q(x) = Ax^2 + Bx + C$ through three test points $x_1 < x_2 < x_3$ with known function values f_1, f_2, f_3 .

SQE Minimum Point Formula

The minimum x^* of the approximating quadratic polynomial is given by:

$$x^* = \frac{1}{2} \frac{(x_2^2 - x_3^2)f_1 + (x_3^2 - x_1^2)f_2 + (x_1^2 - x_2^2)f_3}{(x_2 - x_3)f_1 + (x_3 - x_1)f_2 + (x_1 - x_2)f_3} \quad (19)$$

Alternatively, we can express the calculation using coordinate differences:

$$x^* = x_2 + \frac{1}{2} \frac{(x_2 - x_1)^2(f_2 - f_3) - (x_2 - x_3)^2(f_2 - f_1)}{(x_2 - x_1)(f_2 - f_3) - (x_2 - x_3)(f_2 - f_1)} \quad (20)$$

If x^* is close to x_2 within a tolerance limit, x^* is taken as the optimum. Otherwise, the worst point among $\{x_1, x_2, x_3\}$ is replaced by x^* , and the calculation is repeated.

Quick Revision — Module 2

Single-Variable Optimization | OE-EC704B

Key Formulas & Equations

Necessary Condition	$f'(x^*) = 0$ (stationary point requirement).
Sufficient Condition	$f''(x^*) > 0 \Rightarrow \text{local min}; f''(x^*) < 0 \Rightarrow \text{local max}.$
Interval Halving Width	$L_{new} = \frac{1}{2}L_{old}$ (achieved with 3 evaluations).
Fibonacci Point 1	$x_1 = a + \frac{F_{n-2}}{F_n}(b - a).$
Fibonacci Reduction Ratio	$\frac{L_n}{L_1} = \frac{1}{F_n}$ (where F_n is the n -th Fibonacci number).
SQE Minimum Point	$x^* = x_2 + \frac{1}{2} \frac{(x_2 - x_1)^2(f_2 - f_3) - (x_2 - x_3)^2(f_2 - f_1)}{(x_2 - x_1)(f_2 - f_3) - (x_2 - x_3)(f_2 - f_1)}.$

One-Line Definitions

- **Stationary Point:** A point at which the first derivative of a differentiable function is zero.
- **Inflection Point:** A point on a curve at which the concavity changes, where $f''(x) = 0$ and $f'''(x) \neq 0$.
- **Unimodal Function:** A function that has exactly one local extremum (minimum or maximum) in a given interval.
- **Bracketing Method:** A search algorithm designed to find an initial interval containing the optimum.
- **Region Elimination:** Systematically discarding a sub-interval of search space using unimodality comparisons.
- **Fibonacci Search:** An optimal region-elimination search algorithm utilizing Fibonacci ratios.
- **Successive Quadratic Estimation:** An interpolation algorithm that fits quadratic curves through test coordinates to locate the minimum.

Frequently Asked / Viva Questions

Q1. State the necessary and sufficient conditions for a local minimum of a single-variable function.

Necessary condition: $f'(x^*) = 0$. Sufficient condition: $f''(x^*) > 0$.

Q2. What happens if $f'(x^*) = f''(x^*) = 0$ at a candidate point?

The test is inconclusive. We must evaluate higher-order derivatives. If the first non-zero derivative is of odd order, it is an inflection point. If it is of even order and positive, it is a local minimum.

Frequently Asked / Viva Questions (contd.)

Q3. Define a unimodal function and state why it is important for region-elimination methods.

A function is unimodal if it has only one optimum in a given range. Unimodality guarantees that comparing function values at two points ($f(x_1)$ and $f(x_2)$) yields a valid sub-region to eliminate.

Q4. How does the Interval Halving method work?

It divides the interval into four equal parts using three points ($x_1 < x_0 < x_2$). Comparing their function values allows the elimination of half the interval ($L_{new} = 0.5L_{old}$).

Q5. Why is the Fibonacci search method considered optimal among region-elimination techniques?

For a pre-specified number of evaluations n , it achieves the maximum possible reduction in the interval of uncertainty ($1/F_n$) without requiring any derivative information.

Q6. What is the relation between Fibonacci search and Golden Section search?

Golden Section search is the limiting case of Fibonacci search as the number of evaluations $n \rightarrow \infty$. It uses the constant Golden Ratio $\gamma \approx 0.618$ instead of discrete Fibonacci fractions.

Q7. Write down the quadratic estimation formula for the minimum point x^* .

$$x^* = \frac{1}{2} \frac{(x_2^2 - x_3^2)f_1 + (x_3^2 - x_1^2)f_2 + (x_1^2 - x_2^2)f_3}{(x_2 - x_3)f_1 + (x_3 - x_1)f_2 + (x_1 - x_2)f_3} \text{ for three ordered points } x_1 < x_2 < x_3.$$

Q8. Differentiate between bracketing methods and point estimation methods.

Bracketing methods find an interval containing the optimum (e.g., Exhaustive Search). Point estimation methods fit functional approximations (e.g., quadratic curves) to predict the exact coordinate of the optimum.

Q9. Under what conditions does the successive quadratic estimation (SQE) method converge?

SQE converges rapidly if the objective function is smooth, continuous, and has a unimodal shape resembling a parabola in the neighborhood of the optimum.

Q10. What is the significance of the Fibonacci numbers sequence in search algorithms?

The Fibonacci numbers satisfy the property $F_{n-2}/F_n \rightarrow 0.382$ and $F_{n-1}/F_n \rightarrow 0.618$, which balances the spacing of evaluation points so that previous calculations are reused in subsequent steps.

Module 3

Fundamentals of Gradient-Based Search

1. Fundamentals of Gradient-Based Search

Gradient-based single-variable methods use derivative information to find stationary points by solving:

$$g(x) = f'(x) = 0 \quad (21)$$

These algorithms are highly efficient because derivatives provide the slope and curvature of the objective function, indicating the direction and distance to the optimum.

2. Newton-Raphson Method

The Newton-Raphson method is a second-order method that approximates the function locally with a quadratic Taylor series expansion.

2.1 Derivation of the Iterative Formula

Let x_k be the current estimate of the optimum. Expanding the first derivative $f'(x)$ about x_k using a first-order Taylor series yields:

$$f'(x) \approx f'(x_k) + f''(x_k)(x - x_k)$$

To locate the stationary point (x_{k+1} where $f'(x_{k+1}) = 0$):

$$f'(x_k) + f''(x_k)(x_{k+1} - x_k) = 0$$

Rearranging for x_{k+1} gives the standard **Newton-Raphson Update**:

Newton-Raphson Iterative Equation

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)} \quad (22)$$

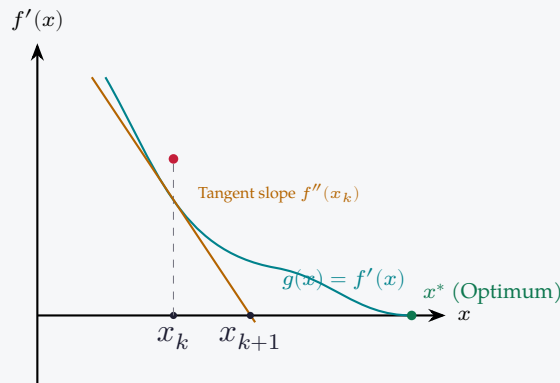
Where:

- $f'(x_k)$ is the first derivative (slope) at step k .
- $f''(x_k)$ is the second derivative (curvature) at step k .

2.2 Convergence and Characteristics

- **Quadratic Convergence:** Near the optimum, the error decreases quadratically ($e_{k+1} \propto e_k^2$), making it the fastest gradient-based method.
- **Limitations:** Requires computing the analytical second derivative $f''(x)$. If $f''(x_k) = 0$, the method fails. It is highly sensitive to the initial guess x_0 and can diverge if starting far from the root.

Graphical projection of Newton-Raphson



3. Bisection Method

The bisection method is a robust first-order bracketing technique that solves $f'(x) = 0$ by halving intervals based on sign changes.

Bisection Criterion

Let the initial interval be $[a, b]$. If $f'(x)$ is continuous and:

$$f'(a) \cdot f'(b) < 0 \quad (23)$$

then by the Intermediate Value Theorem, at least one stationary point x^* exists in $[a, b]$ where $f'(x^*) = 0$.

3.0.1 Algorithm Steps

1. Calculate the midpoint: $x_0 = \frac{a+b}{2}$ and evaluate $f'(x_0)$.
2. If $|f'(x_0)| \leq \epsilon$ (tolerance), stop; $x^* \approx x_0$.
3. If $f'(a) \cdot f'(x_0) < 0$, the root lies in $[a, x_0]$. Set $b = x_0$.
4. If $f'(x_0) \cdot f'(b) < 0$, the root lies in $[x_0, b]$. Set $a = x_0$.
5. Repeat until $|b - a| \leq \epsilon$. Convergence is guaranteed, though linear and slow.

4. Secant Method

The secant method approximates the second derivative $f''(x)$ in the Newton-Raphson formula using two previous evaluations, avoiding analytical differentiation.

4.1 Iterative Formula

The curvature is approximated using finite differences between x_k and x_{k-1} :

$$f''(x_k) \approx \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}} \quad (24)$$

Substituting this into the Newton equation yields the **Secant Iterative Update**:

Secant Method Equation

$$x_{k+1} = x_k - f'(x_k) \left[\frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})} \right] \quad (25)$$

This method requires two initial guess points x_0 and x_1 , and converges superlinearly with a convergence rate of:

$$\text{Rate} \approx 1.618 \quad (26)$$

5. Computer Programs / Pseudo-Codes

Newton-Raphson Optimization Pseudo-code

Input: Initial guess x_0 , tolerance eps , max_iter

Output: Local minimum coordinate x^*

Function NewtonRaphson(x_0 , eps , max_iter):

$x = x_0$

 For $k = 1$ to max_iter :

$\text{df} = \text{FirstDerivative}(x)$

$\text{ddf} = \text{SecondDerivative}(x)$

 If $|\text{ddf}| < 1\text{e-}12$:

 Print "Error: Curvature close to zero!"

 Return null

$x_{\text{new}} = x - \text{df} / \text{ddf}$

 If $|x_{\text{new}} - x| < \text{eps}$ or $|\text{df}| < \text{eps}$:

 Return x_{new}

$x = x_{\text{new}}$

 Return x

Secant Optimization Pseudo-code

Input: Two initial guesses x_0 , x_1 , tolerance ϵ , max_iter

Output: Local minimum coordinate x^*

Function SecantMethod(x_0 , x_1 , ϵ , max_iter):

$x_{\text{prev}} = x_0$

$x_{\text{curr}} = x_1$

 For $k = 1$ to max_iter :

$df_{\text{prev}} = \text{FirstDerivative}(x_{\text{prev}})$

$df_{\text{curr}} = \text{FirstDerivative}(x_{\text{curr}})$

 If $|df_{\text{curr}} - df_{\text{prev}}| < 1e-12$:

 Print "Error: Denominator is zero!"

 Return null

$x_{\text{next}} = x_{\text{curr}} - df_{\text{curr}} * (x_{\text{curr}} - x_{\text{prev}}) / (df_{\text{curr}} - df_{\text{prev}})$

 If $|x_{\text{next}} - x_{\text{curr}}| < \epsilon$:

 Return x_{next}

$x_{\text{prev}} = x_{\text{curr}}$

$x_{\text{curr}} = x_{\text{next}}$

 Return x_{curr}

Quick Revision — Module 3

Gradient-Based Methods | OE-EC704B

Key Formulas & Convergence Rates

Newton-Raphson Formula	$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$
Newton Convergence	Quadratic rate: $e_{k+1} \approx Ce_k^2.$
Bisection Condition	$f'(a) \cdot f'(b) < 0$ (signs must be opposite).
Secant Update Formula	$x_{k+1} = x_k - f'(x_k) \left[\frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})} \right].$
Secant Convergence	Superlinear rate: $e_{k+1} \approx Ce_k^{1.618}.$

One-Line Definitions

- **Gradient-Based Optimization:** Minimizing/maximizing functions using directional slope (derivatives) rather than search comparisons alone.
- **Newton-Raphson Method:** An iterative root-finding technique using tangent lines to estimate the point where the derivative is zero.
- **Bisection Method:** A root-finding algorithm that repeatedly bisects an interval and selects a sub-interval in which a sign change occurs.
- **Secant Method:** A gradient-based optimization method that replaces analytical second derivatives with finite difference slope approximations.
- **Quadratic Convergence:** A convergence behavior where the number of correct decimal places roughly doubles with each iteration.
- **Intermediate Value Theorem:** Theorem stating that if a continuous function has opposite signs at interval boundaries, it must cross zero inside.

Frequently Asked / Viva Questions

Q1. Compare Newton-Raphson and Secant methods based on derivative requirements.

Newton-Raphson requires evaluating both the first derivative $f'(x)$ and the analytical second derivative $f''(x)$ at each step. The Secant method only requires the first derivative $f'(x)$ because it approximates $f''(x)$ using finite differences.

Q2. State the convergence rate of the Newton-Raphson method and define what it means.

It converges quadratically ($e_{k+1} \propto e_k^2$). This means the error in step $k + 1$ is proportional to the square of the error in step k , leading to extremely rapid convergence near the root.

Frequently Asked / Viva Questions (contd.)

Q3. Why does the Newton-Raphson method fail if $f''(x_k) = 0$?

Because the formula involves dividing by the second derivative. If $f''(x_k) = 0$, the tangent to $f'(x)$ becomes parallel to the x-axis, and the projection point x_{k+1} moves to infinity (division by zero error).

Q4. Explain the role of the sign check $f'(a) \cdot f'(b) < 0$ in the Bisection method.

It guarantees that the first derivative $f'(x)$ changes sign across the interval $[a, b]$. For a continuous derivative, this ensures that at least one stationary point (root of $f'(x) = 0$) lies within the interval.

Q5. What are the primary disadvantages of the Bisection method?

It has a slow, linear convergence rate. It does not utilize the magnitude of the derivative, only its sign, making it take many iterations compared to Newton's method.

Q6. Write down the Secant method update equation.

$$x_{k+1} = x_k - f'(x_k) \left[\frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})} \right].$$

Q7. How does the Secant method differ from the regular False Position (Regula Falsi) root-finding method?

False Position always brackets the root and retains points of opposite signs. The Secant method simply uses the last two calculated points, meaning it is not guaranteed to bracket the root and can occasionally diverge.

Q8. What is the convergence rate of the Secant method?

It converges superlinearly with order $\alpha = \frac{1+\sqrt{5}}{2} \approx 1.618$ (the Golden Ratio).

Q9. Under what conditions can the Newton-Raphson method diverge?

It can diverge if the initial guess x_0 is chosen far from the optimum, especially in regions with inflection points, local extrema of $f'(x)$, or flat regions where $f''(x) \approx 0$.

Q10. Write a short MATLAB/Python routine snippet for updating the Secant point.

```
x_next = x_curr - df_curr * (x_curr - x_prev) / (df_curr - df_prev)
where df is the derivative function.
```

Module 4

Optimality Criteria for Multivariable Functions

1. Optimality Criteria for Multivariable Functions

Let $f(\mathbf{x})$ be a continuous multivariable function defined on the design vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$.

1.1 Necessary Condition (First-Order)

Theorem: Multivariable Necessary Condition

If $f(\mathbf{x})$ is differentiable, a necessary condition for a point $\mathbf{x}^*$ to be a local extremum (minimum or maximum) is that the gradient vector evaluated at $\mathbf{x}^*$ must vanish:

$$\nabla f(\mathbf{x}^*) = \mathbf{0} \implies \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]_{\mathbf{x}=\mathbf{x}^*}^T = \mathbf{0} \quad (27)$$

Points satisfying this are called **Stationary Points**.

1.2 Sufficient Condition (Second-Order)

Optimality is determined by evaluating the **Hessian Matrix** $H(\mathbf{x}^*)$, defined as the matrix of second-order partial derivatives:

$$H(\mathbf{x}^*) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]_{n \times n} \quad (28)$$

Theorem: Multivariable Sufficient Condition

At the stationary point $\mathbf{x}^*$:

- If $H(\mathbf{x}^*)$ is **Positive Definite** (all eigenvalues > 0), then $\mathbf{x}^*$ is a **local minimum**.
- If $H(\mathbf{x}^*)$ is **Negative Definite** (all eigenvalues < 0), then $\mathbf{x}^*$ is a **local maximum**.
- If some eigenvalues are positive and others negative, $\mathbf{x}^*$ is a **Saddle Point**.

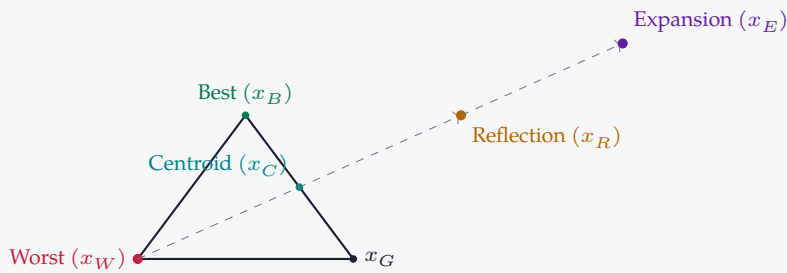
2. Direct Search Methods (No Gradients)

Direct search methods do not compute derivatives, making them suitable for noisy or discontinuous functions.

2.1 Simplex Search Method (Nelder-Mead)

A simplex in n -dimensional space is a geometric figure defined by $n + 1$ vertices (e.g., a triangle in 2D, a tetrahedron in 3D). The method moves the simplex toward the optimum using reflection, expansion, contraction, and shrinkage operations.

Nelder-Mead Simplex Operations in 2D



2.2 Hooke-Jeeves Pattern Search Method

Hooke-Jeeves is a conceptual search method consisting of two primary components:

1. **Exploratory Move:** Performs local searches along the coordinate axes to find a direction of descent.
2. **Pattern Move:** Performs an accelerated step along the direction established by connecting successive base points, bypassing slow orthogonal paths.

3. Gradient-Based Descent Methods

3.1 Cauchy's (Steepest Descent) Method

Steepest descent moves in the direction of the local negative gradient vector, which represents the direction of maximum rate of decrease of the function.

Steepest Descent Formulation

The iterative update equation is:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k) \quad (29)$$

Where:

- $\nabla f(\mathbf{x}_k)$ is the gradient vector at step k .
- α_k is the step length, determined by solving a unidirectional line search problem:

$$\text{Minimize } f(\mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)) \quad \text{w.r.t. } \alpha \quad (30)$$

Characteristics: Simple to implement but exhibits slow convergence near the optimum due to orthogonal zig-zagging trajectories.

3.2 Newton's Multivariable Method

Newton's method uses a second-order quadratic approximation of the objective function.

Newton's Multivariable Update

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [H(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k) \quad (31)$$

Where $[H(\mathbf{x}_k)]^{-1}$ is the inverse of the Hessian matrix.

Characteristics: Converges quadratically (extremely fast near the optimum) but requires calculating and inverting the Hessian matrix at every step, which is computationally expensive for large systems.

4. Multi-Objective and Pareto Optimization

In real-world engineering, we often optimize multiple conflicting objectives simultaneously (e.g., minimizing cost while maximizing safety).

4.1 Pareto Dominance & Optimality

Instead of a single optimum, multi-objective optimization yields a set of trade-off solutions.

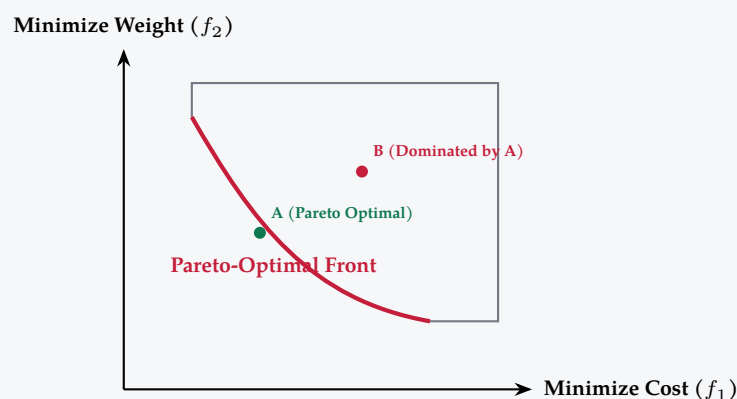
Definition of Pareto Dominance

A decision vector $\mathbf{x}_1$ is said to **dominate** $\mathbf{x}_2$ if:

1. $\mathbf{x}_1$ is no worse than $\mathbf{x}_2$ in all objectives.
2. $\mathbf{x}_1$ is strictly better than $\mathbf{x}_2$ in at least one objective.

A solution is **Pareto Optimal** if it is not dominated by any other feasible solution. The set of all Pareto optimal solutions in the objective space forms the **Pareto Front**.

Pareto Optimal Front (Conflicting Objectives)



Quick Revision — Module 4

Multivariable Optimization | OE-EC704B

Key Concepts & Mathematical Rules

Gradient Vector (∇f)	$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T = \mathbf{0}.$
Hessian Matrix (H)	$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$ (square symmetric matrix).
Cauchy's Update	$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)$ (Steepest Descent).
Newton's Update	$\mathbf{x}_{k+1} = \mathbf{x}_k - H^{-1} \nabla f(\mathbf{x}_k)$ (Quadratic Convergence).
Pareto Dominance Rule	$\mathbf{x}_1 \prec \mathbf{x}_2 \iff \forall i : f_i(\mathbf{x}_1) \leq f_i(\mathbf{x}_2) \text{ and } \exists j : f_j(\mathbf{x}_1) < f_j(\mathbf{x}_2).$

One-Line Definitions

- **Positive Definite Matrix:** A matrix whose eigenvalues are all strictly positive (indicates local minimum structure).
- **Saddle Point:** A stationary point at which the Hessian matrix is indefinite (eigenvalues of mixed signs).
- **Nelder-Mead Simplex:** A derivative-free search method using vertices of a geometric figure to crawl toward the optimum.
- **Hooke-Jeeves Method:** A coordinate-based search method combining local exploratory moves and pattern acceleration.
- **Steepest Descent:** An iterative gradient method moving in the direction of the local negative gradient vector.
- **Pareto Front:** The set of all non-dominated solutions representing optimal trade-offs in multi-objective problems.

Frequently Asked / Viva Questions

Q1. State the necessary and sufficient conditions for a local minimum of a multivariable function.

Necessary condition: The gradient vector must vanish, $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Sufficient condition: The Hessian matrix $H(\mathbf{x}^*)$ must be positive definite.

Q2. What is a Hessian matrix and what is its role in multivariable optimization?

The Hessian is a square symmetric matrix of second-order partial derivatives. Its role is to provide local curvature information to identify if a stationary point is a minimum, maximum, or saddle point.

Q3. Explain why Cauchy's (Steepest Descent) method exhibits a zig-zag search

Frequently Asked / Viva Questions (contd.)

trajectory.

Because successive search directions in Cauchy's method are mathematically orthogonal: $\mathbf{d}_k \cdot \mathbf{d}_{k-1} = 0$. This perpendicular stepping causes the algorithm to bounce back and forth in narrow valleys.

Q4. What are the operations performed in the Nelder-Mead simplex search method?

The operations are Reflection, Expansion, Contraction (Outer and Inner), and Shrinkage.

Q5. Differentiate between exploratory and pattern moves in the Hooke-Jeeves method.

An exploratory move searches along the coordinate axes to find a local descending direction. A pattern move accelerates the search by stepping along the vector connecting successive base points.

Q6. Compare multivariable Newton's method with the Steepest Descent method.

Newton's method converges quadratically (very fast) but requires calculating and inverting the Hessian matrix. Steepest descent only requires the first-order gradient, making it computationally simpler per step but much slower to converge.

Q7. What is Pareto optimality in multi-objective optimization?

A solution is Pareto optimal if no other solution exists that improves one objective without simultaneously degrading at least one other objective.

Q8. Define the concept of a saddle point.

A saddle point is a stationary point where the gradient is zero, but the Hessian is indefinite, meaning the function increases in some directions and decreases in others.

Q9. What is the purpose of a unidirectional line search in multivariable algorithms?

To find the optimal step length α_k that minimizes the function along the selected search direction $\mathbf{d}_k$: $\min_{\alpha} f(\mathbf{x}_k + \alpha \mathbf{d}_k)$.

Q10. What makes a matrix positive definite using Sylvester's Criterion?

A symmetric matrix is positive definite if and only if all its leading principal minors (determinants of top-left submatrices) are strictly positive.

Module 5

Characteristics of Constrained Problems

1. Characteristics of Constrained Problems

Constrained optimization restricts the search space to a feasible region. Let us analyze the constraint boundaries:

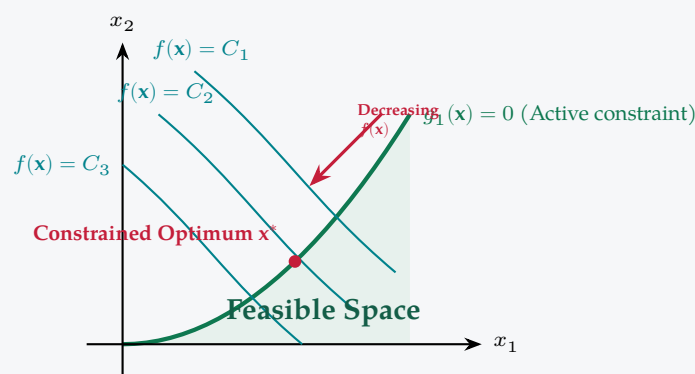
Active vs. Inactive Constraints

At a candidate design point $\mathbf{x}$:

- An inequality constraint $g_j(\mathbf{x}) \leq 0$ is said to be **Active** if $g_j(\mathbf{x}) = 0$. The point lies exactly on the boundary of that constraint.
- It is **Inactive** if $g_j(\mathbf{x}) < 0$. The constraint is satisfied, and the point lies in the interior of the feasible space.
- It is **Violated** if $g_j(\mathbf{x}) > 0$. The design point is infeasible.

All equality constraints $h_k(\mathbf{x}) = 0$ must be active at all feasible points.

Feasible Region and Constrained Optimum



2. Direct Constrained Optimization Methods

Direct methods evaluate only function values and directly handle constraints by adjusting candidate coordinates.

2.1 Box's Complex Method

Box's Complex Method is an extension of the Nelder-Mead Simplex method designed to handle inequality constraints:

- Uses a geometric shape called a **Complex** consisting of $k \geq n + 1$ vertices (typically $k = 2n$).
- If a generated reflection point violates an inequality constraint, it is moved halfway toward the centroid of the remaining feasible points until it becomes feasible.

2.2 Kelly's Cutting Plane Method

Kelly's Cutting Plane Method solves non-linear convex constrained problems:

1. Linearizes all non-linear constraints using first-order Taylor expansions to form a linear programming (LP) relaxation.
2. Solves the LP relaxation. If the solution is infeasible for the original non-linear constraints, it adds a new linear inequality constraint (a **Cutting Plane**) that slices off the infeasible LP solution.
3. Repeats the process until convergence, generating a sequence of tightening linear hulls.

3. Indirect Methods (Transformation Techniques)

Transformation techniques reformulate a constrained problem into an equivalent sequence of unconstrained problems.

3.1 Exterior Penalty Function Method

The exterior penalty method starts search in the infeasible region and penalizes constraint violations, forcing iterations toward the feasible boundary.

Exterior Penalty Function Formulation

The unconstrained penalty objective function Φ is defined as:

$$\Phi(\mathbf{x}, r_p) = f(\mathbf{x}) + r_p \sum_{j=1}^m [\max(0, g_j(\mathbf{x}))]^2 + \frac{1}{r_p} \sum_{k=1}^p [h_k(\mathbf{x})]^2 \quad (32)$$

Where r_p is a positive penalty parameter.

Algorithm:

1. Select an initial penalty value $r_p^{(1)}$ (usually small) and minimize $\Phi(\mathbf{x}, r_p^{(1)})$ unconstrained.
2. Increase the parameter: $r_p^{(k+1)} = c \cdot r_p^{(k)}$ (where $c > 1$).
3. Re-minimize Φ . As $r_p \rightarrow \infty$, the sequence of unconstrained minima converges to the constrained optimum $\mathbf{x}^*$.

3.2 Interior Penalty (Barrier) Function Method

The interior penalty method starts inside the feasible region and creates a barrier at the boundary that prevents iterations from escaping into the infeasible space.

Interior Barrier Function Formulations

The unconstrained barrier objective function Φ is defined using one of two barriers:

- **Inverse Barrier Function:**

$$\Phi(\mathbf{x}, r_k) = f(\mathbf{x}) - r_k \sum_{j=1}^m \frac{1}{g_j(\mathbf{x})} \quad (33)$$

- **Logarithmic Barrier Function:**

$$\Phi(\mathbf{x}, r_k) = f(\mathbf{x}) - r_k \sum_{j=1}^m \ln [-g_j(\mathbf{x})] \quad (34)$$

Where r_k is the positive barrier parameter.

Algorithm:

1. Start at a strictly feasible point. Minimize $\Phi(\mathbf{x}, r_k^{(1)})$.
2. Decrease the parameter: $r_k^{(k+1)} = d \cdot r_k^{(k)}$ (where $0 < d < 1$).
3. Re-minimize. As $r_k \rightarrow 0$, the barrier effect weakens at the boundary, allowing convergence to the boundary optimum $\mathbf{x}^*$.

4. Convex Optimization Methods

Convex optimization is a special class of optimization with unique mathematical guarantees.

Convex Programming Problem

A constrained optimization problem is a **Convex Programming Problem** if:

1. The objective function $f(\mathbf{x})$ is convex.
2. All inequality constraints $g_j(\mathbf{x}) \leq 0$ are convex functions.
3. All equality constraints $h_k(\mathbf{x}) = 0$ are linear (affine) functions.

Theorem: For a convex programming problem, **any local minimum is guaranteed to be the global minimum**, eliminating the risk of getting trapped in local sub-optimal states.

Quick Revision — Module 5

Constrained Optimization | OE-EC704B

Key Concepts & Formulations

Active Inequality	$g_j(\mathbf{x}^*) = 0$ (optimum lies on the constraint boundary).
Inactive Inequality	$g_j(\mathbf{x}^*) < 0$ (constraint is satisfied with slack).
Exterior Penalty Φ	$\Phi = f(\mathbf{x}) + r_p \sum [\max(0, g_j)]^2$ (where $r_p \rightarrow \infty$).
Inverse Interior Barrier	$\Phi = f(\mathbf{x}) - r_k \sum \frac{1}{g_j}$ (where $r_k \rightarrow 0$).
Logarithmic Barrier	$\Phi = f(\mathbf{x}) - r_k \sum \ln(-g_j)$ (where $r_k \rightarrow 0$).
Convex Definition	$f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$ for $\lambda \in [0, 1]$.

One-Line Definitions

- **Active Constraint:** A constraint whose boundary passes directly through the current design point.
- **Complex Method:** A constrained direct search method utilizing $2n$ vertices to explore bounded spaces.
- **Transformation Technique:** Reformulating constrained problems into sequential unconstrained formulations.
- **Exterior Penalty Method:** A transformation technique that approaches the boundary from the infeasible design region.
- **Interior Barrier Method:** A transformation technique that approaches the boundary from strictly within the feasible region.
- **Convex Function:** A function where a line segment connecting any two points on its graph lies above or on the graph.

Frequently Asked / Viva Questions

Q1. What is the difference between active and inactive constraints at an optimum point?

An active constraint satisfies $g_j(\mathbf{x}^*) = 0$ (lies on the boundary). An inactive constraint satisfies $g_j(\mathbf{x}^*) < 0$ (lies inside the boundary) and does not limit the local optimum.

Q2. Compare the search paths of the Exterior Penalty and Interior Barrier methods.

The exterior penalty method starts from an infeasible point and moves inward toward the feasible boundary as $r_p \rightarrow \infty$. The interior barrier method starts from

Frequently Asked / Viva Questions (contd.)

a strictly feasible point and moves outward toward the boundary as $r_k \rightarrow 0$.

Q3. Why does the Interior Barrier method require a strictly feasible starting point?

Because the barrier terms (such as $1/g_j$ or $\ln(-g_j)$) are mathematically undefined or return complex/imaginary values in the infeasible region ($g_j(\mathbf{x}) > 0$).

Q4. State Sylvester's conditions for a function to be strictly convex.

A function is strictly convex if its Hessian matrix $H(\mathbf{x})$ is positive definite (all eigenvalues > 0) at all points in the design space.

Q5. What is the main advantage of Convex Programming?

Any local minimum of a convex programming problem is guaranteed to be the global minimum.

Q6. Explain the concept behind Kelly's Cutting Plane method.

It linearizes the non-linear constraints to solve an approximating linear program. It then iteratively adds linear constraint inequalities (cutting planes) to cut off infeasible areas.

Q7. Why are equality constraints challenging for the Interior Barrier method?

Because equality constraints $h_k(\mathbf{x}) = 0$ define a region with zero thickness, leaving no "interior" space to start a strictly feasible search path.

Q8. How does Box's Complex method handle constraint violations?

If a generated point is infeasible, it is iteratively retracted halfway toward the centroid of the remaining feasible points until it becomes feasible.

Q9. Write down the logarithmic barrier objective function expression.

$$\Phi(\mathbf{x}, r_k) = f(\mathbf{x}) - r_k \sum_{j=1}^m \ln(-g_j(\mathbf{x})).$$

Q10. What is the physical meaning of the penalty parameter r_p in the exterior penalty method?

It acts as a scaling coefficient for constraint violations. As $r_p \rightarrow \infty$, the penalty for violating constraints becomes extremely large, forcing the unconstrained optimizer to find a feasible solution.

Module 6

Genetic Algorithm (GA)

1. Genetic Algorithm (GA)

Genetic Algorithms are global meta-heuristic search algorithms inspired by Charles Darwin's theory of natural evolution (survival of the fittest). They operate on a population of candidate designs represented as chromosomes.

1.1 Basic Genetic Operators

A standard GA processes a population using three core operators:

1. Selection (Reproduction): Identifies and copies high-fitness individuals to a mating pool.

- *Roulette Wheel Selection:* Probability of selecting individual i is proportional to its fitness f_i :

$$p_i = \frac{f_i}{\sum_{j=1}^N f_j} \quad (35)$$

- *Tournament Selection:* Randomly selects a few individuals and chooses the best.

2. Crossover (Recombination): Combines genetic material from two parent chromosomes to create offspring.

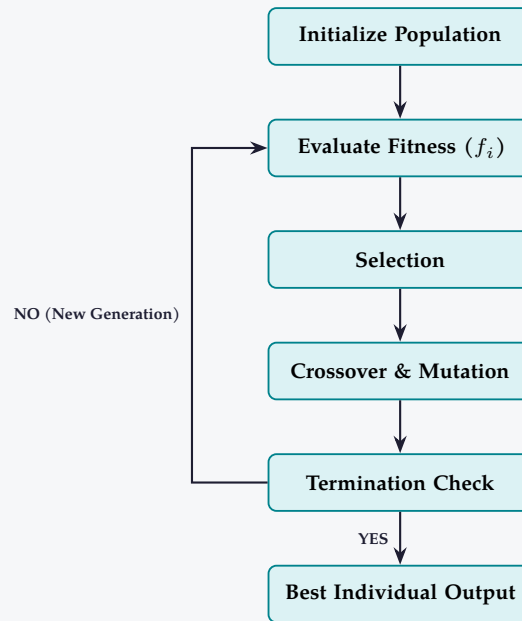
- *Single-Point Crossover:* Parents are split at a random point and swapped.
- *Multi-Point Crossover:* Multiple split boundaries are defined.

3. Mutation: Randomly flips bits or values in a chromosome with a small probability ($p_m \approx 0.005$) to maintain diversity and prevent premature convergence.

1.2 The Genetic Algorithm Flowchart

The iterative lifecycle of a GA is mapped below:

Genetic Algorithm Search Loop



2. Simulated Annealing (SA)

Simulated Annealing is inspired by the physical process of annealing in metallurgy, where a metal is heated to a high temperature and cooled slowly to minimize internal energy, forming a perfect crystal lattice.

2.1 The Metropolis Acceptance Criterion

SA allows uphill moves (accepting worse designs) to escape local minima. Let the change in objective value be $\Delta E = f(\mathbf{x}_{new}) - f(\mathbf{x}_{old})$.

Metropolis Criterion Equations

- If $\Delta E < 0$ (improvement), the new state is **always accepted**: $P(Accept) = 1$.
- If $\Delta E \geq 0$ (worse state), the new state is accepted with a **Boltzmann probability**:

$$P(Accept) = \exp\left(-\frac{\Delta E}{T}\right) \quad (36)$$

Where T is the virtual temperature.

As the temperature T is slowly reduced using a cooling schedule (e.g., $T_{k+1} = \alpha T_k$ with $0.8 \leq \alpha < 1.0$), the probability of accepting worse moves approaches 0, and the algorithm converges to the global minimum.

3. Swarm Intelligence Algorithms

Swarm intelligence algorithms mimic the collective behavior of decentralized, self-organized biological swarms.

3.1 Particle Swarm Optimization (PSO)

PSO is inspired by bird flocking. A swarm of particles moves through the design space, where each particle tracks its own personal best position ($\mathbf{p}_{best,i}$) and the swarm's global best position ($\mathbf{g}_{best}$).

PSO Particle Update Equations

At generation $t + 1$, the velocity vector $\mathbf{v}_i$ and position vector $\mathbf{x}_i$ of particle i are updated as:

$$\mathbf{v}_i^{(t+1)} = w\mathbf{v}_i^{(t)} + c_1r_1(\mathbf{p}_{best,i} - \mathbf{x}_i^{(t)}) + c_2r_2(\mathbf{g}_{best} - \mathbf{x}_i^{(t)}) \quad (37)$$

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)} \quad (38)$$

Where:

- w is the **inertia weight** regulating momentum.
- c_1 (cognitive parameter) and c_2 (social parameter) are acceleration constants.
- r_1, r_2 are random numbers uniformly distributed in $[0, 1]$.

3.2 Swarm Variants: DE, BFA, and ACO

- **Differential Evolution (DE):** A vector-based evolutionary algorithm that creates new candidate vectors by adding weighted differences between random population vectors to a target vector, followed by crossover and selection.
- **Bacterial Foraging Algorithm (BFA):** Mimics the chemotaxis (swimming and tumbling), reproduction, and dispersal behaviors of *E. coli* bacteria.
- **Ant Colony Optimization (ACO):** Mimics the foraging behavior of real ants, which communicate path lengths using chemical pheromones.

Ant Colony Pheromone Update Rule

The pheromone level τ_{ij} on path segment (i, j) is updated after each iteration as:

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \sum_{k=1}^m \Delta\tau_{ij}^k \quad (39)$$

Where:

- ρ is the **pheromone evaporation rate** ($0 < \rho \leq 1$), preventing infinite accumulation.
- $\Delta\tau_{ij}^k$ is the pheromone deposited by ant k : $\Delta\tau_{ij}^k = \frac{Q}{L_k}$ if ant k traversed segment (i, j) (where L_k is the total path length, and Q is a constant).

Quick Revision — Module 6

Advanced Swarm Algorithms | OE-EC704B

Key Concepts & Swarm Metrics

Selection Probability	$p_i = f_i / \sum f_j$ (Roulette Wheel).
Boltzmann Probability	$P(\text{Accept}) = \exp(-\Delta E/T)$ (Simulated Annealing).
PSO Velocity Update	$\mathbf{v}_i^{(t+1)} = w\mathbf{v}_i^{(t)} + c_1r_1(\mathbf{p}_{best} - \mathbf{x}_i) + c_2r_2(\mathbf{g}_{best} - \mathbf{x}_i)$.
PSO Position Update	$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)}$.
ACO Pheromone Update	$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum \Delta\tau_{ij}^k$ (ρ is evaporation rate).

One-Line Definitions

- **Meta-heuristic Algorithm:** A high-level search heuristic designed to find, generate, or select a search path to solve global optimization problems.
- **Crossover Operator:** A GA operator that recombines parts of two parent chromosomes to produce offspring.
- **Elitism:** A GA technique that copies the best-performing chromosomes directly to the next generation without changes.
- **Annealing Cooling Schedule:** A function detailing how the virtual temperature T is systematically reduced during Simulated Annealing.
- **Inertia Weight (w):** A PSO parameter controlling the influence of a particle's historical velocity on its current trajectory.
- **Chemotaxis:** The direct movement of E. coli bacteria along chemical gradients, consisting of alternating swims and tumbles.

Frequently Asked / Viva Questions

Q1. Explain the three primary operators used in standard Genetic Algorithms.

The operators are: Selection (copies high-fitness chromosomes to the mating pool), Crossover (recombines genetic sequences from parent pairs to create offspring), and Mutation (randomly alters genes to maintain population diversity).

Q2. How does Roulette Wheel selection work?

In Roulette Wheel selection, the selection probability of an individual is proportional to its fitness value. Highly fit individuals occupy a larger slice of the virtual wheel, increasing their chances of being chosen.

Q3. What is simulated annealing and what is its thermodynamic analogy?

Frequently Asked / Viva Questions (contd.)

SA is a global optimization algorithm. Its thermodynamic analogy is the slow cooling (annealing) of a molten metal to achieve a low-energy, highly structured crystal lattice, which corresponds to the global minimum.

Q4. Why does Simulated Annealing accept worse designs, and how does it prevent divergence?

It accepts worse designs to climb out of local minima traps. It prevents divergence by reducing the acceptance probability ($e^{-\Delta E/T}$) as the virtual temperature T cools, making the search purely local at the end.

Q5. Write down the velocity update equation for Particle Swarm Optimization (PSO).

$$\mathbf{v}_i^{(t+1)} = w\mathbf{v}_i^{(t)} + c_1r_1(\mathbf{p}_{best,i} - \mathbf{x}_i^{(t)}) + c_2r_2(\mathbf{g}_{best} - \mathbf{x}_i^{(t)}).$$

Q6. Define the cognitive and social terms in the PSO update equations.

The cognitive term ($c_1r_1(\mathbf{p}_{best} - \mathbf{x})$) pulls the particle toward its own personal historical best. The social term ($c_2r_2(\mathbf{g}_{best} - \mathbf{x})$) pulls the particle toward the global best found by the entire swarm.

Q7. What is the role of pheromone evaporation in Ant Colony Optimization (ACO)?

It systematically decreases the pheromone levels on all paths ($1 - \rho$). This prevents the algorithm from getting stuck in early sub-optimal paths by allowing alternative pathways to compete.

Q8. How does the Differential Evolution (DE) algorithm generate new candidate vectors?

It creates new trial vectors by adding a weighted difference vector between two random population members to a third vector, followed by crossover and selection.

Q9. List the three operational steps of the Bacterial Foraging Algorithm (BFA).

Chemotaxis (swimming and tumbling), Reproduction (splitting healthy bacteria), and Elimination and Dispersal (dispersing bacteria randomly to model environmental shocks).

Q10. What is Elitism in Genetic Algorithms and why is it used?

Elitism is the process of copying the absolute best individual(s) directly to the next generation without crossover or mutation. This prevents the loss of the best found design due to random operators.